



Exact and heuristic approaches to Truck–Drone Delivery Problems

Júlia C. Freitas^{*}, Puca Huachi V. Penna, Túlio A.M. Toffolo

Federal University of Ouro Preto, Department of Computing, Brazil

ARTICLE INFO

Keywords:

Unmanned Aerial Vehicle
Traveling Salesman Problem
Drone delivery
Mixed-integer Programming
General Variable Neighborhood Search

ABSTRACT

Collaborative delivery employing drones in last-mile delivery has been an extensively studied topic in recent years. In this paper, it is studied Truck–Drone Delivery Problems (TDDPs) in which a traditional delivery truck is gathered with a drone to cut delivery times and costs. The vehicles work together in a hybrid operation involving one drone launching from a larger vehicle that operates as a mobile depot and a recharging platform. The drone launches from the truck with a single package to deliver to a customer. Each drone must return to the truck to recharge batteries, pick up another package, and launch again to a new customer location. This work proposes a novel Mixed Integer Programming (MIP) formulation and a heuristic approach to address the problem. The proposed MIP formulation yields better linear relaxation bounds than previously proposed formulations for all instances, and was capable of optimally solving several unsolved instances from the literature. A hybrid heuristic based on the General Variable Neighborhood Search metaheuristic combining Tabu Search concepts is employed to obtain high-quality solutions for large-size instances. The efficiency of the algorithm was evaluated on 1415 benchmark instances from the literature, and over 80% of the best known solutions were improved.

1. Introduction

Unmanned Aerial Vehicles (UAVs), generally known as drones, have recently shown a lot of potential in a wide range of civil purposes. In the last years, with the evolution of drone technology, these vehicles and their applications have been attracting the attention of both the academic literature and large transportation companies (Mercer, 2018). Drones are employed within many applications in different fields. They are no longer limited to military use, and different businesses are now investing in these devices for faster and more responsive customer service. One of the most important purposes for these unmanned vehicles lies in disaster management, rescue operations and health-care. Drones can operate in dangerous environments that are inaccessible to humans. Another potential use concerns law enforcement, given these devices have the innate ability to hover around locations without drawing much attention from people. Thus, surveillance and public safety are also prominent applications of drones. Drone delivery is another important and innovative application of drones, one which has become particularly relevant with the social distancing requirements due to the Covid-19 pandemic (Guillot, 2020; Douglas, 2020). The potential of drone delivery is vast both concerning operational cost and customer service efficiency. However, these aerial vehicles are not a replacement for the traditional delivery trucks, due to their low payload capacity and short flying range. Drones can, however, be a useful complementary feature to the delivery process since they are

not limited by road networks. The applications of unmanned vehicles spread far beyond the preceding examples. For a complete survey on drone applications and insights into emerging modeling approaches, the reader is referred to Otto et al. (2018).

This work focuses on a groundbreaking delivery system composed by a truck and drone working collaboratively named Truck–Drone Delivery Problem (TDDP). We approach two different TDDPs: the Flying Sidekick Traveling Salesman Problem (FSTSP), first introduced by Murray and Chu (2015), and the Traveling Salesman Problem with Drone (TSP-D), presented by Agatz et al. (2018). Both problems are a generalization of the Traveling Salesman Problem (TSP), one of the most well known and studied problems in operational research. While the classical TSP consists in finding an optimal route for one vehicle to deliver goods to a set of customers (Fig. 1(a)), the TDDP consists in finding optimal routes for both a drone and one traditional delivery vehicle to perform goods distribution (Fig. 1(b)). In Fig. 1, circular nodes indicate customers served by the truck while triangular nodes represent those served by the drone. Dashed and continuous lines indicate a drone trip and a truck tour, respectively. The delivery truck departs from the depot carrying a drone and all customer parcels. As the driver performs deliveries, the drone is launched from the truck, carrying the parcel for an individual customer. While the drone is on a trip, no intervention from the delivery driver is required. After

^{*} Corresponding author.

E-mail addresses: julia.freitas@aluno.ufop.edu.br (J.C. Freitas), puca@ufop.edu.br (P.H.V. Penna), tulio@toffolo.com.br (T.A.M. Toffolo).

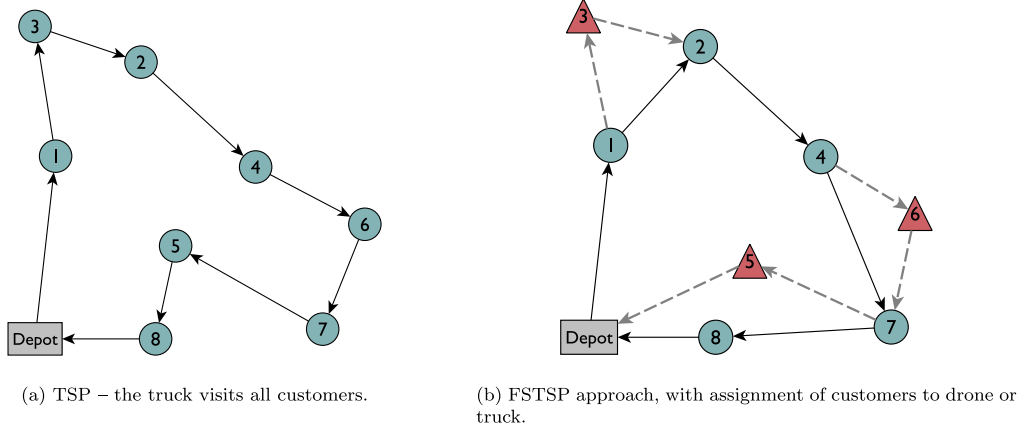


Fig. 1. Customer deliveries are made by either a traditional delivery truck or via drone.

delivering the parcel, the drone returns to the truck in a new customer location. One of the advantages of the truck–drone delivery system is the efficiency enhancement, as the drone’s capability to reach more customers increases once it is launched from the truck closer to the customer delivery location.

The contribution of this paper is twofold. A Mixed Integer Programming (MIP) formulation is proposed for the problem, capable of solving several unsolved instances from the literature. Also, we addressed a heuristic capable of finding high-quality solutions to the larger instances. Both the MIP model and the proposed heuristic were designed to tackle the two variants of drone delivery problems addressed by [Murray and Chu \(2015\)](#) and [Agatz et al. \(2018\)](#). The heuristic improved the best-known solution of 1138 instances from the literature (80.4% of the total). These results show the improvements that may occur in the total delivery time when using specific neighborhood structures of the problem within the local search. The results also indicate that combining truck and drone for last-mile parcel delivery results in huge improvements in delivery time over truck tours for the instances considered.

In summary, this work presents both exact and heuristic approaches for TDDP, resulting in improvements over the best known bounds and solutions for a vast number of problem instances from the literature.

This manuscript is organized as follows. Section 2 details the problem. Section 3 presents a few drone applications and a literature overview concerning TDDP. A novel MIP formulation for the problem is proposed in Section 4. The heuristic algorithm developed is introduced and described in Section 5. Section 6 presents the computational experiments and, finally, Section 7 summarizes the conclusions.

2. Problem description

The Flying Sidekick Traveling Salesman Problem (FSTSP) presented by [Murray and Chu \(2015\)](#) can be described as follows. Let $G = (V, A)$ be an undirected graph with $|V| = n + 1$ nodes. Node $v_0 \in V$ represents the depot, where drone and truck must depart from and return to exactly once. The two vehicles (drone and truck) may depart and return from the depot either in tandem or independently. While traveling in tandem, the drone is transported by the truck to conserve battery. Every other node $v \in V \setminus \{v_0\}$ represents a customer that must be visited exactly once. Some customers must be served by the truck due to the travel time being superior to drone endurance, while others may be served by either the truck or the drone.

The vehicles do not necessarily follow the same distance metric. The truck is limited to the road network, while the drone can use a different network to travel between customers.

During the delivery, the drone may make multiple trips, each composed of three locations. The trip begins at the *launch* node, which can

be either the depot or a customer location. Before launch, the setup time s^L is required for the drone to have its battery changed and the parcel loaded. The second node in a trip is the *delivery* node, which represents a customer serviced by the drone. The final or *return* node is the location where the drone is collected. This node may be either the depot or a location visited by the truck. An additional setup time s^R is required for the *return* node since the drone must be recovered. We consider that whenever the drone is launched, the package will be successfully delivered and the drone will return to the truck or the depot without any issue, and within the drone’s flight endurance limit. If the drone trip ends at the depot, the drone cannot be re-launched.

The drone can only be recovered at a different position than the launch location in the truck route. Therefore, the drone must not be launched multiple times from the same location. The drone may visit only one customer per trip. The truck may serve other customers in between launch and return locations, while the drone is serving a customer. Moreover, either of the two vehicles may have to wait for the other one depending on their arrival times at the return node. The waiting time must not exceed the drone’s battery power. Regarding vehicle capacity, the drone can carry and deliver only one parcel per trip, while no limit is imposed on the truck’s capacity.

The FSTSP objective is to minimize the time required to complete all deliveries and return both vehicles to the depot.

3. Related literature

This section provides a literature review of the potential benefit of pilotless technology. This review first presents a few promising drone applications. Next, it aims at distinguishing the different approaches and delivery problem with drones.

Delivery applications have recently received considerable media attention, mainly because of the prospect of door-to-door express deliveries at low-cost. With this new concept, delivery times and costs could be significantly reduced. Because of the overwhelming amount of announcement in drone research by the industry, the literature has been growing significantly. Hereafter, we present some approaches that address the FSTSP and similar problems.

[Murray and Chu \(2015\)](#) proposed a mixed integer linear programming formulation for two delivery-by-drone problems in which the delivery is performed by truck and drone. In the Parallel Drone Scheduling Traveling Salesman Problem (PDSTSP) the drone attends customers within flight range of the depot while the truck attends the remaining. The operations occur independently, i.e., while the truck follows a TSP route the drones fly to the customers and back to the depot multiple times. In the other problem, the truck and drone work collaboratively (Flying Sidekick Traveling Salesman Problem — FSTSP). The authors implemented a simple and effective heuristic approach for

both problems. A similar work is the one by [Bouman et al. \(2018\)](#). They addressed an exact solution approach for the TSP-D based on dynamic programming that can solve problems up to 16 customers. This number of customers is larger than the mathematical programming approaches presented in the literature thus far. [Ponza \(2016\)](#) based his dissertation on the FSTSP proposed by [Murray and Chu \(2015\)](#). He proposed a slightly different mathematical formulation to the problem and presented an analysis of several heuristics that could be used to resolve the problem. Moreover, he introduced a new set of instances to the literature.

[Dell'Amico et al. \(2020\)](#) approached the PDTSP proposing a MILP model and several matheuristics. The authors experiment with the algorithms on the benchmark instances introduced by [Saleu et al. \(2018\)](#) and [Murray and Chu \(2015\)](#). The computational study validates that the proposed algorithms produce competitive results in terms of both efficiency and effectiveness mainly on small and medium-size instances.

[Wang et al. \(2017\)](#) study the Vehicle Routing Problem with Drones (VRPD) from a worst-case point of view. The paper describes several theorems formulation for the vehicle routing problem with drones and represents bounds on maximal savings to the companies. [Poikonen et al. \(2017\)](#) expand the description of the theorems comparing different drone configurations in the delivery process to determine the maximum benefit. For example, the trade-off between speed and the number of drones, i.e., they compare what is better, a more substantial number of slower drones or a smaller number of faster drones.

While the works above aim to minimize the time required to complete the tour, in [Ha et al. \(2018\)](#) the objective is to minimize the total operational cost of a drone–truck delivery system. They proposed a MILP formulation to the problem and a heuristic called TPS-LS, both inspired on the work of [Murray and Chu \(2015\)](#). Furthermore, a GRASP heuristic was presented. The results showed that GRASP provides better solution quality while TPS-LS deliver a lower solution quality, yet very quickly.

Another approach for the drone–truck system is the one presented in [Jeong et al. \(2019\)](#). They considered two practical issues to evaluate the problem, the effect of parcel weight on drone energy consumption and restricted flying areas. Their sensitivity analysis shows that the increase in package weight and no-fly zones reduce the efficient use of drones as it limits their flight range, especially when the two factors are combined.

The FSTSP proposed by [Gonzalez-R et al. \(2020\)](#) allows the truck to wait for the drone where it was launched. The drone also can perform multiple visits per launch. The authors considered drone energy, i.e., the battery is changed between drone trips and it is considered fully charged after the swap. An iterative greedy search heuristic combined with simulated annealing was proposed.

[de Freitas and Penna \(2020\)](#) introduces new instances based on the TSPLIB and compares the HGVNS (General Variable Neighborhood Search) result with instances found in the literature ([Agatz et al., 2018](#); [Ponza, 2016](#)). In this work, we complement the heuristics by using a list, based on the Tabu Search, to avoid cycling in the neighborhoods. Here, we also propose a MILP to solve the FSTSP in the instances proposed in [Murray and Chu \(2015\)](#), [Ponza \(2016\)](#).

The multiple flying sidekicks traveling salesman problem (mFSTSP) introduced by [Murray and Raj \(2020\)](#) considers a delivery truck operating in coordination with a fleet of drones. The drones are launched from the truck to deliver a single package, then return to the truck where it can be loaded again. They employed a three-phased (I. initial truck assignments, II. create drone routes, III. combining phase I and II) heuristic solution to approach the problem. The heuristic result analysis revealed that drones with high-speed and long-range offer greater benefits in larger geographic regions, where customers are distributed over a larger area.

[Ferrandez et al. \(2016\)](#) introduced a truck–drone delivery system where multiples drones travel per truck. They investigated the time and energy associated with a truck–drone delivery network compared to a

standalone truck or drone delivery. Besides, they proposed a k-means and a genetic algorithm to determine the optimal number of launch locations and drones per truck. Following the multiple drones per truck approach, [Karak and Abdelghany \(2019\)](#) present a mathematical formulation and solution methodology for the hybrid vehicle–drone routing problem (HVDRP) for pick-up and delivery services. The problem is formulated as a mixed-integer program, which minimizes the vehicle and drone routing cost to serve all customers. The formulation captures the vehicle–drone routing interactions during the drone dispatching and collection processes and accounts for drone operation constraints related to flight range and load carrying capacity limitations. A novel solution methodology is developed which extends the classic Clarke and Wright algorithm (HCWH) to solve the HVDRP. The performance of the developed heuristic is benchmarked against two other heuristics, namely, the vehicle-driven routing heuristic (VDH) and the drone-driven routing heuristic (DDH). A set of experiments are conducted to evaluate the performance of the developed heuristics. While the VDH and the DDH focus on optimizing the cost of one mode only, the HCWH is shown to outperform these two heuristics in terms of minimizing the cost of the entire multi-modal network. The network operation cost is shown to be minimum when the used drones are balanced in terms of their flight range and load carrying capacity.

[Schermer et al. \(2019\)](#), [Wang et al. \(2017\)](#), [Poikonen et al. \(2017\)](#), [Ulmer and Thomas \(2018\)](#), [Pugliese and Guerriero \(2017\)](#) tackled the Vehicle Routing Problem with Drones (VRPD). In [Schermer et al. \(2019\)](#), they formulated the VRPD as a Mixed Integer Linear Program (MILP), and introduced several sets of valid inequalities aiming to improve the performance of solvers. To address large instances, they presented a matheuristic approach that exploits the problem structure of the VRPD. The authors proposed the Drone Assignment and Scheduling Problem (DASP) defined as minimization problem that given an existing routing of trucks, looks for an optimal assignment and schedule of drones such that the makespan is minimized.

Contrasting with the others approaches, [Song et al. \(2018\)](#) addressed a delivery problem where drones serve all customers. The authors described the Unmanned Aerial Vehicle Routing Problem (UAVRP) presenting new features to drone delivery problem: capacity and time window. Multiple drones, package weight impacting the vehicle battery life and customers demand to be satisfied are also considered. Service stations are strategically positioned to respect the endurance of drones and minimize delivery time. In these stations vehicles can be recharged and reloaded; thus, the drones can serve customers persistently. Computation experiments evaluated a heuristic and a mixed integer linear programming (MILP) formulation. While the MILP formulation could not solve large-scale problems, the heuristic successfully derives optimal or near-optimal solutions for them in a short time. Concerning drone-only delivery ([Dorling et al., 2017](#)) presented in their work an energy consumption model for multicopter drones and provided a linear approximation for it. They proposed the Drone Delivery Problem (DDP) which seeks to minimize cost (MC-DDP) or delivery time (MT-DDP) while considering battery weight, payload weight, and drone reuse. They proposed a MILP implementation and a Simulated Annealing (SA) to solve practical scenarios with hundreds of locations. Comparing the approaches, the SA implementation consistently finds near-optimal solutions to problems with eight or fewer locations. The heuristic behavior in larger instances with 125 to 500 customers showed consistent results. [Arenzana et al. \(2020\)](#) work presented a strategic framework to quantify the efficiency of hospital operations. They introduced a MILP to design drone delivery network for hospital deliveries. The problem minimizes drone traveling time, battery consumption levels, vehicle investment, and infrastructure costs. The trajectories designed between hospitals conform to the latest air traffic management regulations. They presented a case study based in London. The paper shows that drones present numerous advantages in comparison with traditional road transport. With operational costs averaging 30% depending on drone model and operational

Table 1

Summary of the main features of FSTSP contributions in the literature.

Problem class	#Drones	#Trucks	Same L/R	Same net	Endurance	Capacity	Drone multiple visits	Sync	Related Work
FSTSP	1	1	×	×	×	×	×	✓	Murray and Chu (2015) Ponza (2016) Ha et al. (2018) de Freitas and Penna (2020)
FSTSP	1	1	×	×	×	×	✓	✓	Gonzalez-R et al. (2020)
PDSTSP	n	1	×	×	×	×	×	×	Murray and Chu (2015) Dell'Amico et al. (2020)
TSP-D	1	1	✓	✓	×	×	×	✓	Bouman et al. (2018) Agatz et al. (2018)
VRPD	n	m	×	✓	×	×	×	✓	Wang et al. (2017) Poikonen et al. (2017) Schmer et al. (2019) Ulmer and Thomas (2018) Pugliese and Guerriero (2017)
VRPDR	n	m	✓	×	✓	✓	×	✓	Dayarian et al. (2020)
VRPDR	1	1	×	×	✓	✓	×	✓	Dayarian et al. (2020)
DDP	n	0	✓	–	✓	✓	×	–	Dorling et al. (2017)
UAVRP	n	0	✓	–	✓	✓	✓	–	Song et al. (2018)
FSTSP-ECNZ	1	1	✓	×	✓	✓	×	✓	Jeong et al. (2019)
mFSTSP	n	1	×	×	×	✓	×	✓	Murray and Raj (2020)
HVDRP	n	1	✓	✓	×	✓	✓	✓	Karak and Abdelghany (2019)
STRPD	n	1	✓	×	×	✓	✓	✓	Moshref-Javadi et al. (2020)

parameters, such as vehicle range and payload size, the drone-based model increases service reliability (lower variability in travel time) and overcomes initial investment.

Table 1 summarizes the related work mentioned in this section. Column *Problem class* defines the problem approached in the paper. #Drones and #Trucks describe, respectively, the number of drones and trucks considered in the problem. Column Same L/R indicates the node where the drone is launched is necessarily equal from the node it returns. Same net describes if the network the vehicles travel are the same. Then, there are three columns describing drones: (1) whether energy consumption is evaluated (Endurance), (2) whether capacity is considered (Capacity), (3) whether drone can perform multiple visits (Drone multiple visits). Column Sync indicates if trucks and drones perform a coordinated operation. Finally, the last column reports the works that studied the problem.

4. Formulation

This section proposes a compact Mixed Integer Programming (MIP) formulation for the FSTSP. As with the problem description in Section 2, a graph $G = (V, A)$ is considered to represent the problem. **Table 2** presents the notation employed throughout the formulation. Note in this table that L is the set of possible *positions* for the truck to visit a customer. This set contains indices used to select the order that customers are visited, which is crucial to synchronize the truck and the drone. It is defined such that $L = \{0, \dots, n\}$, where n is the number of customers.

Differently from the formulations proposed by Ponza (2016) and Murray and Chu (2015), this formulation considers three variable sets that uses an additional position index:

- t^ℓ : variable that defines the total travel time until position ℓ ;
- $x_{i,j}^\ell$: binary variable equal to 1 if the truck traverses arc (i, j) at position ℓ , and 0 otherwise;
- $y_{i,k,j}^{\ell,\ell'}$: binary variable equal to 1 if the drone traverses arcs (i, k) and (k, j) , launching from vertex i at position ℓ and returning to the truck in vertex j at position $\ell' > \ell$, and 0 otherwise.

Table 2

Sets and input data utilized within the formulation.

V	Vertex set including the depot and the n customers, $V = \{v_0, \dots, v_n\}$
V'	Vertex set excluding the depot, $V' = V \setminus \{v_0\}$
A	Arc set
D	Set of possible drone paths (i, k, j) formed by two arcs, (i, k) and (k, j) that respects the drone's maximum endurance
L	Set of possible order positions for truck visits, $L = \{0, \dots, n\}$
e	Drone flight endurance time
s^L	Setup time for launching the drone
s^R	Setup time for returning the drone
$\tau_{i,j}$	Time required by the truck to traverse arc (i, j)
$\tau_{i,k,j}^D$	Time required by the drone to traverse arcs (i, k) and (k, j)
M	Upper bound for the time required by the truck to visit all customers

At first sight, the number of variables may seem prohibitively significant. However, in practice, this number can be considerably reduced by filtering variable sets x and y to consider only feasible connections, meaning only $(i, j) \in A$ and $(i, k, j) \in D$ should be considered. Moreover, despite requiring more variables, the formulation here proposed is stronger than those proposed by Murray and Chu (2015) and Ponza (2016), yielding better linear relaxation lower bounds for all instances considered (computational results are presented in Section 6).

The formulation is presented by Eqs. (1)–(15). To simplify the notation and reduce the constraints length, we assume $x_{i,j}^\ell = 0$ for all $(i, j) \notin A$ and, analogously, $y_{i,k,j}^{\ell,\ell'} = 0$ for all $(i, k, j) \notin D$ and all nonexistent position pairs (ℓ, ℓ') with $\ell \geq \ell'$. Note that such variables are not generated by our implementation, whose source code is available online.¹

$$\min. \quad t^{n+1} \quad (1)$$

$$s.t. \quad \sum_{j \in V} x_{v_0,j}^0 = \sum_{j \in V} \sum_{\ell \in L \setminus \{0\}} x_{j,v_0}^\ell = 1 \quad (2)$$

$$\sum_{j \in V} \sum_{\ell \in L} x_{i,j}^\ell = \sum_{j \in V} \sum_{\ell \in L} x_{j,i}^\ell \leq 1 \quad \forall i \in V \quad (3)$$

¹ The formulation's implementation is available at <http://www.github.com/tuliotoffolo/fstsp>.

$$\sum_{j \in V} x_{j,k}^{\ell-1} = \sum_{j \in V} x_{k,j}^{\ell} \quad \forall k \in V', \ell \in L \setminus \{0\} \quad (4)$$

$$\sum_{(i,j) \in A} x_{i,j}^{\ell} \leq 1 \quad \forall \ell \in L \quad (5)$$

$$\sum_{(i,k,j) \in D} \sum_{l=0}^{\ell} \sum_{l'=l+1}^n y_{i,k,j}^{l,l'} \leq 1 \quad \forall \ell \in L \quad (6)$$

$$\sum_{j \in V} \sum_{\ell \in L} x_{k,j}^{\ell} + \sum_{i \in V'} \sum_{j \in V} \sum_{\ell \in L} \sum_{\ell' \in L} y_{i,k,j}^{\ell,\ell'} = 1 \quad \forall k \in V' \quad (7)$$

$$\sum_{k \in V'} \sum_{j \in V} \sum_{\ell' \in L} y_{i,k,j}^{\ell,\ell'} \leq \sum_{j \in V} x_{i,j}^{\ell} \quad \forall i \in V, \ell \in L \quad (8)$$

$$\sum_{i \in V} \sum_{k \in V'} \sum_{\ell \in L} y_{i,k,j}^{\ell,\ell'} \leq \sum_{i \in V} x_{i,j}^{\ell'-1} \quad \forall j \in V, \ell' \in L \quad (9)$$

$$t^{\ell'} - t^{\ell} \leq e + M \left(1 - \sum_{(i,k,j) \in D} y_{i,k,j}^{\ell,\ell'} \right) \quad \forall \ell \in L \setminus \{0\}, \quad \ell' \in L : \ell' > \ell \quad (10)$$

$$\begin{aligned} t^{\ell} &\geq t^{\ell-1} + \sum_{(i,j) \in A} \tau_{i,j} x_{i,j}^{\ell-1} \\ &+ \sum_{(i,k,j) \in D, \ell'=\ell} \sum_{\ell'>1}^n s^L y_{i,k,j}^{\ell-1,\ell'} \\ &+ \sum_{(i,k,j) \in D} \sum_{l=1}^{\ell-1} s^R y_{i,k,j}^{l,\ell} \end{aligned} \quad \forall \ell \in L \setminus \{0\} \cup \{n+1\} \quad (11)$$

$$t^{\ell'} \geq t^{\ell} + \sum_{(i,k,j) \in D} \left(s^L + \tau_{i,k,j}^D + s^R \right) y_{i,k,j}^{\ell,\ell'} \quad \forall \ell' \in L \setminus \{0\}, \quad \ell \in L : \ell < \ell' \quad (12)$$

$$t^0 = 0 \quad (13)$$

$$x_{i,j}^{\ell} \in \{0, 1\} \quad \forall (i, j) \in A, \ell \in L \quad (14)$$

$$y_{i,k,j}^{\ell,\ell'} \in \{0, 1\} \quad \forall (i, j, k) \in D, \ell \in L, \quad \ell' \in L : \ell' > \ell \quad (15)$$

The objective function presented by Eq. (1) minimizes the total time to visit all customers, given by the sum of the truck's traveling time and all required setup times to launch and collect the drone. Constraints (2) ensure the truck leaves the depot at position zero and returns to it at the tour's end. Constraints (3) limit the number of truck visits to any customer to one. Constraints (4) are flow preservation constraints which force the truck to leave a customer at the subsequent position of its visit. Constraints (5) limit the number of arcs traversed at each position to at most one. Constraints (6) prohibit launching the drone more than once in overlapping time windows (given by l and l') and therefore assert the drone is not launched when it is not with the truck. Note that l and l' are used to cover all time windows, including position ℓ . Constraints (7) guarantee every customer is visited exactly once, either by the truck or by the drone. Constraints (8) and (9) synchronize the truck's position with the drone's launch and return, respectively. Constraints (10) certify the drone's endurance is respected. Note that these constraints employ a 'Big M', which disables the constraint whenever the drone is not launched. The value of M is set to an upper bound² on the time at which both the drone and the truck return to the depot. Also, the truck's travel time is not considered for endurance when the drone launches from the depot (when $\ell = 0$). Constraints (11) update the travel time until position ℓ considering the truck's route. Setup times s^L and s^R of launching and returning the drone, respectively, are also added if the drone launches from the truck at position $\ell - 1$ or if it returns to the truck at position ℓ . Similarly, Constraints (12) ensure the travel time until position ℓ includes the time traveled by the drone and eventual setup times s^L and s^R . Therefore, time t^{ℓ} of any position $\ell > 0$

considers the travel time of both truck and drone, including whichever is larger. Constraint (13) sets the total travel time at the first position to zero and, finally, Constraints (14) and (15) declare the binary nature of variables x and y .

5. The hybrid heuristic

The proposed heuristic algorithm, named Hybrid Tabu General Variable Neighborhood Search (HTGVNS), is a hybrid metaheuristic that combines the exact solution of a TSP solver and the exploration capabilities of systematical neighborhood changes. The HTGVNS employs the Route First Cluster Second approach of Beasley (1983) in which first a TSP is solved and then clusters are created by assigning customers to the drone.

The following sections detail the heuristic approach. First Section 5.1 introduces the different neighborhood structures used within the algorithm. Then, Section 5.2 formally introduces the proposed heuristic.

5.1. Neighborhood structures

Several *neighborhood structures* were developed to explore the search space of the FSTSP, based on different *moves*. When a move is applied on a reference solution S , a *neighbor solution* S' is obtained. A neighborhood structure N consists of all possible applications of a *move*. Considering a reference solution S , this results in a set of neighbor solutions $N(S)$, which is hereinafter referred to as *neighborhood*.

The neighborhood structures considered are based mostly on classical TSP moves, with some of them relying on specificities of the problem. The solution space is visited applying the Best Improvement (BI) (Hansen et al., 2016) approach, which exhaustively explores a neighborhood and returns the solution with the lowest objective value, i.e., the best neighbor.

Only feasible solutions are accepted within the proposed algorithm, meaning that infeasible neighbors are discarded. Note that to be feasible a route (i, k, j) must respect the drone battery's life, i.e. $\tau_{i,k,j}^D \leq e$. Similarly, the drone battery power must endure until the truck arrival at the *return* node. Sub-routes are considered to avoid a drone launch before a return, as illustrated by Fig. 2, where each rectangle represents a sub-route. In Fig. 2(a), drone customers can be assigned only to sub-routes 1 and 3. A launch must not occur in sub-route 2 since a drone trip is already associated with this sub-route. It is important to note that for every new drone trip, the truck route splits into one more sub-route. Therefore, new launches can only occur in sub-routes not associated with a drone. Fig. 2(b) describes the split of sub-route 3 as the result of a new drone trip.

The reader is directed to de Freitas and Penna (2020) for more details on the neighborhood structures considered here, which are divided into two categories: intra-route and inter-route, detailed next.

5.1.1. Intra-route neighborhood structures

The intra-route neighborhood structures considered are based on classical TSP moves.

- i. SWAP(1,1): It swaps one customer in the solution with another one.
- ii. SWAP(2,1): It swaps two consecutive customers in the solution with another one.
- iii. SWAP(2,2): It swaps two consecutive customers in the solution with other two consecutive ones.
- iv. 2-OPT: This move is the classical 2-opt move proposed by Croes (1958). Two edges are removed, and then, the two paths created are reconnected in the only possible way to keep a valid tour.
- v. REINSERTION: This move consists of removing a customer from its current position and reinserting it in another one.

² The upper bound was obtained by a simple Nearest Neighbor constructive heuristic, and its value is available as part of Ponza (2016) instances.

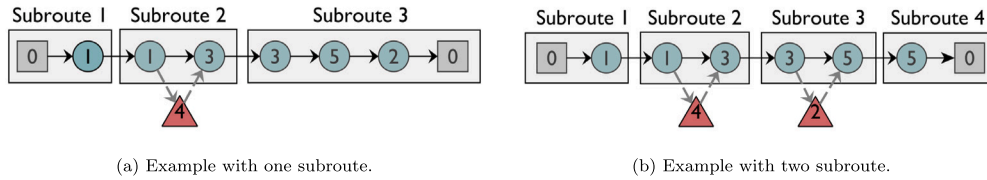


Fig. 2. Customer deliveries are made by either a traditional delivery truck or via drone.



Fig. 3. Swap truck-drone customer move.



Fig. 4. Swap truck-drone customer move.

- vi. OR-OPT2: This move consists of removing two consecutive customers from their current position and reinserting them in another one.

5.1.2. Inter-route neighborhood structures

The inter-route neighborhood structures are based on moves which envision solution improvement by exploring the problem characteristics.

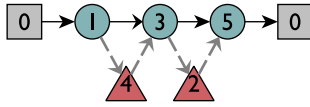
- i. SHIFT(1,0): This move consists of removing one truck customer and subsequently inserting it into the drone route. This move requires a cubic time, $\mathcal{O}(c^2c')$ where c is the number of truck customers and c' is the number of eligible customers not currently assigned to the drone. This complexity time refers to three nested loops where the first defines the launch node i , subsequently, the delivery node j and, lately, the return node k . This candidate trip must not exceed drone endurance. The pair of nodes i and k are not necessarily adjacent, but i must precede k . The selected combination of $\langle i, j, k \rangle$ is the one that presents the more considerable decreasing cost of the truck route within the removal of customer j (see Fig. 3).
- ii. SWAP(1,1): This swap move consists of simply swapping two customers in the solution. One customer belongs to the truck and the other to the drone route. The complexity is $\mathcal{O}(c'c)$ where c' and c are the number of drone and truck customers, respectively. This complexity is due to the necessity of comparing all drone nodes with every truck customer to find a combination at least as good as the previous or has a lower objective value. The swap is only performed if the drone endurance is not violated, i.e., the new drone trip can be completed before the drone runs out of battery power and the truck sub-route does not exceed the drone flight limit (see Fig. 4).
- iii. SWAP(0,1): This move is used to shake the current solution by turning a drone customer into a truck customer. A customer is removed from the drone route and inserted into the truck route in a position that generates the least impact on the solution quality. This move may increase the objective value, as it adds another customer to the truck route. However, if the truck waiting time

is larger than the drone trip, a decrease in the objective value may occur. This move requires combining drone customers with every position of the truck route to get the customer position that produces the smallest impact on the solution quality. Therefore, the time complexity of performing this move is $\mathcal{O}(c'c)$ where c' and c are the number of drone and truck customers, respectively (see Fig. 5).

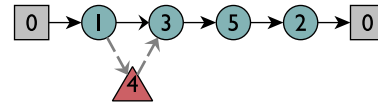
5.2. HTGVNS algorithm

The proposed algorithm first builds an optimal TSP route by employing the exact approach from Applegate et al. (2016). This results in a truck route with all customers, including eligible drone customers, which is used to build an initial solution to the FSTSP. This procedure, presented by Algorithm 1, requires two arguments: (i) an initial TSP solution S and (ii) the eligible drone customers C' . The algorithm iterates over the eligible drone customers until no improvement occurs. For each customer, $j \in C'$ the cost of removing it from the truck's route is computed (line 5). It is also verified for each subroute if it is associated with a drone trip (line 6). For example, in Fig. 2(a) Subroute 1 is not associated with a drone trip while Subroute 2 is. If the subroute is paired with a drone, an attempt is made to insert customer j into the truck's route between adjacent nodes i and k (line 8). Otherwise, the cost of serving customer j by drone is computed (line 10). Each pair of nodes (i, k) such that i precedes k in a given subroute that is not currently connected with the drone is investigated. The goal is to compute the travel time associated with the drone trip launching from node i , visiting node j , and finally returning to the truck at node k .

The proposed algorithm is a General Variable Neighborhood Search (GVNS), which consists of Variable Neighborhood Search (VNS) using Variable Neighborhood Descent (VND) as local search. Hansen et al. (2010) introduced the GVNS metaheuristic that relies upon a local search followed by perturbations to escape from local optima. Furthermore, to avoid cycling, it was adopted a tabu list likely in Tabu Search (Glover and Laguna, 1998) to forbid solutions that possess some attributes of recently explored solutions, especially between insertion and removal of drone customers. For example, in Fig. 2(a) customer 4



(a) Original route.



(b) Remove drone customer 2 then reinsert it in the truck route.

Fig. 5. Remove drone customer move.

Algorithm 1: CreateInitialSolution

Input : Eligible drone customer C' and optimal TSP solution S

```

1  $truckSubRoutes \leftarrow \{S\}$ 
2 stop  $\leftarrow$  false
3 while stop = false do
4   forall  $j \in C'$  do
5      $savings \leftarrow$  calculate the savings of removing customer  $j$  from
       the truck,s route
6     forall  $subroute \in truckSubRoutes$  do
7       if  $subroute$  is paired with drone then
8          $cost\_truck \leftarrow$  calculate the cost of relocating customer  $j$ 
           in the truck,s route
9       else
10         $cost\_drone \leftarrow$  calculate the cost of serving customer  $j$ 
          with the drone
11    $savings\_truck \leftarrow savings - cost\_truck$ 
12    $savings\_drone \leftarrow savings - cost\_drone$ 
13   if  $savings\_truck < savings\_drone$  then
14     update solution  $S$  by moving customer  $j$  to the drone,s route
15   else if  $savings\_truck > savings\_drone$  then
16     update solution  $S$  route by relocating customer  $j$  in the
       truck,s route
17   else
18     stop  $\leftarrow$  true
19 return  $S$ 

```

was just added to the drone route as a *drone customer*. Thus, it must not return to the truck route for the next $|TL|$ iterations, where TL is the tabu list.

The Randomized Variable Neighborhood Descent (RVND) is an adaptation of the classic VND heuristic in which the neighborhood's selecting order is randomized. The VND is a heuristic described in Hansen et al. (2016) and further discussed by Souza et al. (2010) and Penna et al. (2013), who showed that the randomized procedure often outperforms the deterministic approach. The RVND procedure developed for the FSTSP is presented in Algorithm 2. The algorithm begins by filling the neighborhood structure list $\mathcal{N}$ (line 1) and initializing solutions S^* and S' (line 2). Next, variable k is initialized (line 3), representing the current number of neighborhoods analyzed. The main loop (line 4) first randomly selects a neighborhood structure (lines 5–8). This structure can be chosen from two different sets, $\mathcal{N}$ or $\mathcal{N} \setminus \mathcal{N}_{TL}$. The first set is selected if the current solution S' is not in tabu list TL . Otherwise, the second set is selected. Afterwards, a new neighboring solution is generated (line 9). If this solution is at least as good as the previous or has a lower objective value than the considered entry is accepted, the counter k is reset to 1 and list $\mathcal{N}$ is shuffled (lines 10–13). Otherwise, the value of k is incremented (lines 14–15). Once the maximum number of neighbors is reached, the best solution produced is returned (line 16).

The combination of RVND and GVNS is presented in Algorithm 3. Three arguments are required: (i) an initial solution S , (ii) the maximum number of interactions and (iii) static tabu neighborhood list $\mathcal{N}_{TL}$.

Algorithm 3 begins by initializing solution S^* , counter k , perturbation level ρ and tabu list TL (lines 1–4). The main loop (line 5) first generates a random neighbor solution S from neighborhood $\mathcal{N}^{(k)}$

Algorithm 2: Randomized Variable Neighborhood Descent

Input: Solution S , tabu list TL , tabu neighborhood structures $\mathcal{N}_{TL}$

```

1 Initialize and shuffle the list  $\mathcal{N}$  of neighborhood structures;
2  $S^* \leftarrow S' \leftarrow S$ 
3  $k \leftarrow 1$ 
4 while  $k \leq |\mathcal{N}|$  do
5   if  $S' \in TL$  then
6      $N \leftarrow$  random neighborhood structure from  $\mathcal{N} \setminus \mathcal{N}_{TL}$ 
7   else
8      $N \leftarrow$  neighborhood structure  $\mathcal{N}^{(k)}$ 
9    $S' \leftarrow$  best neighbor solution in  $N(S)$ 
10  if  $f(S') < f(S^*)$  then
11     $S^* \leftarrow S'$ 
12     $k \leftarrow 1$ 
13    shuffle  $\mathcal{N}$ 
14  else
15     $k \leftarrow k + 1$ 
16 return  $S^*$ 

```

(line 6). If neighborhood $\mathcal{N}^{(k)}$ is in $\mathcal{N}_{TL}$, then solution S is inserted in tabu list TL (lines 7–8). Afterwards, the RVND algorithm is called (line 9). If the solution produced by RVND is better than the current best solution, the best solution is updated (lines 10–11). Note that counter k and perturbation level ρ are reset only if S is an improving solution over S^* (lines 12–13). Otherwise, the value of k is incremented (lines 14–15). Afterwards, the perturbation is executed (lines 16–17), which corresponds to applying neighborhood $\mathcal{N}^{(9)}$ (see Section 5.1) ρ times to the current solution. Next, the perturbation level ρ is increased (line 18) and the loop repeated. Variable ρ is reset when it equals ρ_{max} . Once the main loop reaches its stopping criterion, the best solution is returned (line 19).

Algorithm 3: General Variable Neighborhood Search

Input: Initial solution S , maximum interaction k_{max} , maximum perturbation level ρ_{max} and static tabu neighborhood list $\mathcal{N}_{TL}$

```

1  $S^* \leftarrow S$ 
2  $k \leftarrow 1$ 
3  $\rho \leftarrow 0$ 
4  $TL \leftarrow \emptyset$ 
5 while  $k \leq k_{max}$  do
6    $S \leftarrow$  random neighbor solution in  $\mathcal{N}^{(k)}(S)$ 
7   if  $\mathcal{N}^{(k)} \in \mathcal{N}_{TL}$  then
8     insert  $S$  in list  $TL$ 
9    $S \leftarrow$  RVND( $S, TL, \mathcal{N}_{TL}$ )
10  if  $f(S) < f(S^*)$  then
11     $S^* \leftarrow S$ 
12     $k \leftarrow 1$ 
13     $\rho \leftarrow 0$ 
14  else
15     $k \leftarrow k + 1$ 
16  for  $i = 0$  to  $\rho$  do
17     $S \leftarrow$  random neighbor solution in  $\mathcal{N}^{(9)}(S)$ 
18   $\rho \leftarrow (\rho \bmod \rho_{max} + 1)$ 
19 return  $S^*$ 

```

Table 3

Average number of variables and constraints per instance-set and endurance value.

Instance set	#nodes	e	This work		Ponza (2016)		Murray and Chu (2015)	
			#Vars	#Constrs	#Vars	#Constrs	#Vars	#Constrs
Ponza (2016)	5	1,440	658	158	116	68	–	–
Ponza (2016)	6	1,440	1,023	212	144	547	–	–
Ponza (2016)	7	1,440	1,893	274	196	790	–	–
Ponza (2016)	8	1,440	4,411	344	281	1,125	–	–
Ponza (2016)	9	1,440	4,161	422	295	1,439	–	–
Ponza (2016)	10	1,440	9,208	508	422	1,941	–	–
Murray and Chu (2015)	10	1,200	31,758	453	–	–	867	2,840
Murray and Chu (2015)	10	2,400	41,273	453	–	–	867	2,840

Table 4

Formulation results for Murray and Chu (2015) instances.

Inst.	$e = 20$					$e = 40$				
	M&C (2015)		This Work			M&C (2015)		This Work		
	Sol.	Time	LB ⁰	Sol.	Time	Sol.	Time	LB ⁰	Sol.	Time
A-v1	56.47	1800.15	38.78	⊗ 56.47	10.73	52.10	1800.15	31.93	⊗ 50.57	1305.59
A-v2	53.21	1800.15	36.05	⊗ 53.21	10.07	47.31	1800.15	27.82	⊗ 47.31	671.41
A-v3	53.69	1800.18	37.48	⊗ 53.69	10.71	53.69	1800.18	30.89	⊗ 53.69	925.35
A-v4	67.46	1800.15	51.77	⊗ 67.46	7.30	66.49	1800.15	43.66	⊗ 66.49	675.25
A-v5	50.55	1800.22	30.67	⊗ 50.55	455.30	45.84	1800.22	30.67	⊗ 44.84	764.31
A-v6	47.60	1800.23	27.69	⊗ 47.31	330.01	47.60	1800.23	27.68	⊗ 43.60	569.86
A-v7	51.89	1800.24	30.85	⊗ 48.58	69.80	46.62	1800.24	30.84	⊗ 46.62	491.20
A-v8	64.69	1800.23	43.60	⊗ 61.38	56.82	59.78	1800.23	43.58	⊗ 59.42	677.58
A-v9	45.98	1800.25	30.62	⊗ 42.42	92.00	42.42	1800.25	30.62	⊗ 42.42	211.23
A-v10	43.09	1800.28	27.60	⊗ 41.73	100.72	41.73	1800.28	27.60	⊗ 41.73	157.21
A-v11	48.21	1800.25	30.81	⊗ 42.90	19.26	42.90	1800.25	30.81	⊗ 42.90	142.32
A-v12	61.57	1800.27	43.54	⊗ 55.70	28.37	55.70	1800.27	43.54	⊗ 55.70	102.14
B-v1	49.43	1800.15	28.23	⊗ 49.43	26.08	48.72	1800.15	28.22	⊗ 46.89	543.02
B-v2	50.71	1800.15	28.22	⊗ 50.71	20.48	46.42	1800.15	28.20	⊗ 46.42	137.92
B-v3	56.10	1800.17	35.00	⊗ 56.10	21.50	53.93	1800.17	34.99	⊗ 53.93	575.93
B-v4	69.90	1800.14	49.00	⊗ 69.90	19.42	68.40	1800.14	47.76	⊗ 68.40	640.12
B-v5	45.36	1800.22	28.17	⊗ 43.53	44.28	46.59	1800.22	28.17	⊗ 43.53	101.52
B-v6	44.08	1800.22	27.93	⊗ 43.95	40.88	44.08	1800.22	27.93	⊗ 43.81	69.44
B-v7	51.92	1800.22	34.94	⊗ 49.42	43.36	49.20	1800.22	34.94	⊗ 49.20	86.90
B-v8	65.62	1800.22	47.74	⊗ 62.22	39.82	62.27	1800.22	47.74	⊗ 62.00	38.63
B-v9	44.25	1800.27	28.15	⊗ 42.53	62.33	44.25	1800.27	28.15	⊗ 42.53	36.83
B-v10	43.08	1800.27	27.81	⊗ 43.08	60.98	43.08	1800.27	27.81	⊗ 43.08	43.33
B-v11	49.20	1800.27	34.93	⊗ 49.20	35.41	49.20	1800.27	34.93	⊗ 49.20	71.60
B-v12	62.00	1800.27	47.73	⊗ 62.00	54.01	62.00	1800.27	47.73	⊗ 62.00	46.88
C-v1	69.59	1800.16	54.27	⊗ 69.59	4.07	57.25	1800.16	31.29	⊗ 55.49	1062.18
C-v2	72.15	1800.14	58.45	⊗ 72.15	4.92	58.05	1800.14	36.29	⊗ 58.05	920.71
C-v3	77.34	1800.13	65.44	⊗ 77.34	1.90	69.17	1800.13	49.20	⊗ 68.43	436.34
C-v4	90.14	1800.16	78.59	⊗ 90.14	3.26	82.70	1800.16	62.13	⊗ 82.70	384.01
C-v5	63.25	1800.22	33.55	⊗ 53.05	32.20	53.45	1800.22	30.92	⊗ 51.93	1801.24
C-v6	64.70	1800.24	36.81	⊗ 55.21	61.71	52.33	1800.24	36.29	⊗ 52.33	488.63
C-v7	67.77	1800.21	51.37	⊗ 64.41	34.90	60.74	1800.21	49.09	⊗ 60.74	86.58
C-v8	83.70	1800.20	64.35	⊗ 77.21	32.95	74.69	1800.20	61.89	⊗ 72.97	61.09
C-v9	59.32	1800.23	30.92	⊗ 45.93	170.79	47.25	1800.23	30.92	⊗ 45.93	261.44
C-v10	61.24	1800.23	36.29	⊗ 46.93	32.20	48.87	1800.23	36.29	⊗ 46.93	48.91
C-v11	67.43	1800.23	49.09	⊗ 56.40	19.65	56.40	1800.23	49.09	⊗ 56.40	23.48
C-v12	83.70	1800.22	61.89	⊗ 69.20	9.25	69.20	1800.22	61.89	⊗ 69.20	16.04

6. Experimental analysis

In this section we present experimental results and analysis of the proposed exact and heuristic approaches. The MIP formulations were modeled using the Python-MIP package and solved using the commercial solver Gurobi 9.0 with the default configuration, while the heuristic algorithm was coded in C++ and compiled with G++ 5.3.1. All experiments were executed on Intel Core i7 3.60 GHz computers with 16 GB of RAM running Ubuntu Linux 16.04.

First, Section 6.1 presents and analyzes the results obtained by the proposed MIP formulation considering the instances from Murray and Chu (2015) and Ponza (2016).³ Next, Section 6.2 discusses HTGVNS

results for different benchmarks from the literature. Section 6.2.1 considers the instances proposed by Ponza (2016), Section 6.2.2 considers those by Agatz et al. (2018) and, finally, the set introduced by de Freitas and Penna (2020) is considered in Section 6.2.3.

6.1. Formulation results

The proposed formulation models the problem addressed by Ponza (2016). This problem considers, however, slightly different constraints than those considered by Murray and Chu (2015). There are two points of attention:

1. in the problem described by Murray and Chu (2015), the truck's travel time between the drone's launch and return can be longer than the drone endurance; the drone can therefore run out of battery while waiting for the truck;

³ Ponza's instances can be found at: <https://github.com/tuliotoffolo/fstsp>.

Table 5
Formulation results for Ponza (2016) instances.

Instance	Ponza (2016)		This Work		
	Sol.	Time	LB ⁰	Sol.	Time
Instance_005.1	4456.83	0.13	3851.22	⊗4456.83	0.38
Instance_005.2	3507.07	0.12	1984.71	⊗3507.07	0.07
Instance_005.3	3275.69	0.14	2979.03	⊗3275.69	0.12
Instance_005.4	5312.47	0.09	3423.66	⊗5312.47	0.07
Instance_005.5	5510.17	0.10	5021.23	⊗5510.17	0.05
Instance_006.1	7080.94	0.25	6064.16	⊗7080.94	0.08
Instance_006.2	6147.96	0.32	5713.98	⊗6147.96	0.23
Instance_006.3	6835.16	0.23	5878.56	⊗6835.16	0.08
Instance_006.4	4402.08	0.32	3424.12	⊗4402.08	0.41
Instance_006.5	5392.08	0.38	4031.53	⊗5392.08	0.34
Instance_007.1	5533.85	3.31	3606.98	⊗5533.85	0.48
Instance_007.2	5342.68	1.79	3258.57	⊗5342.68	0.96
Instance_007.3	7725.89	1.07	6293.13	⊗7725.89	0.21
Instance_007.4	7610.38	1.39	6284.05	⊗7610.38	0.16
Instance_007.5	7010.99	2.10	6211.52	⊗7010.99	0.27
Instance_008.1	6709.02	5.90	4764.75	⊗6709.02	1.26
Instance_008.2	6587.18	10.08	4916.63	⊗6587.18	2.06
Instance_008.3	5780.12	14.68	4133.14	⊗5780.12	3.00
Instance_008.4	6505.12	8.91	3694.07	⊗6505.12	1.76
Instance_008.5	5953.51	15.72	4748.36	⊗5953.51	2.48
Instance_009.1	7338.77	189.38	5773.50	⊗7338.77	2.95
Instance_009.2	6204.63	129.12	4073.60	⊗6204.63	3.30
Instance_009.3	7698.14	87.45	3995.14	⊗7698.14	5.16
Instance_009.4	6817.72	79.71	4281.48	⊗6817.72	3.69
Instance_009.5	7802.67	115.02	5253.94	⊗7802.67	4.85
Instance_010.1	5986.71	1800.15	4502.21	⊗5986.71	50.82
Instance_010.2	6394.39	1800.15	5141.80	⊗6394.39	15.81
Instance_010.3	6310.60	1800.15	3204.10	⊗6310.60	21.94
Instance_010.4	8377.92	752.87	7186.84	⊗8377.92	3.58
Instance_010.5	8934.41	1800.15	5662.50	⊗8934.41	10.56

- The formulation proposed by Murray and Chu (2015) do not consider the setup time for launching the drone as part of the drone's flying time, and it does not count for the total completion time or the battery's endurance, even when the drone leaves from the depot.

Formulation (1)–(15) can be adapted to obtain results comparable with those by Murray and Chu (2015) by removing Constraints (10) and altering Constraints (11) and (12). Due to these small differences in the problem definition, we report results for instances by Ponza (2016) and Murray and Chu (2015) separately.

The formulation given by (1)–(15) has a total of $\mathcal{O}(|V|^5)$ variables. However, as aforementioned, the number of variables generated is proportional to the sizes of sets A and D (see Section 6.1), which are generally much smaller in practice than $|V|^2$ and $|V|^3$, respectively.

Table 3 presents the average number of generated variables (#Vars) and constraints (#Constrs) for the instances considered. For comparison, the number of variables and constraints are also presented for the previous formulations proposed by Ponza (2016) and Murray and Chu (2015). The formulation we propose results in a larger number of variables but a smaller number of constraints for most instances. Note how the proposed formulation's dimensions depend heavily on the endurance of the drone (e). This is expected since a smaller endurance enables reducing set D 's size. It is also noteworthy how the number of variables actually within the model is not prohibitive for small instances.

Table 4 presents the results obtained by the altered formulation considering the instances from Murray and Chu (2015) with $e = 20$ and $e = 40$. For compactness, we refer to Murray and Chu (2015) as M&C (2015), to instance 20140810T123437 as A, 20140810T123440 as B and 20140810T123443 as C. Column LB⁰ presents the value of the linear relaxation, column *Sol.* presents the solution value and column *Time* reports the total execution runtime in seconds. Note that a ⊗ is

included next to column *Sol.* whenever the solution is proven optimal by the solver using the indicated formulation. Note also that a runtime limit of 1800 s was imposed and that we omitted column LB⁰ for the formulation proposed by Murray and Chu (2015), since it obtained value zero for all instances.

The proposed formulation resulted in proven optimal solutions for all instances considering Table 4, taking 61 s on average. This result is quite remarkable since with the formulation proposed by Murray and Chu (2015) the solver was incapable of providing any proven optimal solution within the runtime limit.

Table 5 presents the results obtained by Formulation (1)–(15) without alterations considering Ponza (2016)'s smaller instances containing from 5 to 10 customers. No formulation was capable of solving larger instances with 50, 100, 150 and 200 customers. For these instances, the formulations presented by Ponza (2016) did not obtain any feasible solution and the formulation we propose could not be executed due to memory limitations. It is thus by no means a coincidence that these large instances have only been addressed with heuristic approaches so far. Therefore, we developed the HTGVNS to tackle the large instances found in the literature.

The formulations proposed by Ponza (2016) also obtained linear relaxation lower bounds of value zero for all instances, and so we omitted column LB⁰. It is clear that the formulations previously proposed in the literature are outperformed by the one we propose, which was capable of producing significantly better lower bounds and by consequence proven optimal solutions for all small instances, which are represented by a ⊗ in the table.

6.2. HTGVNS results

In this section, solutions provided by HTGVNS for different benchmarks and the parameters of the heuristic are detailed.

The HTGVNS tabu list size $|TL|$ and perturbation level ρ_{max} were manually tuned. After a significant number of experiments considering different values for the parameters, ρ_{max} was defined as $\left\lceil n \times \frac{1}{10} \right\rceil$, with n being the number of customers. Two sets were selected based on their performance to be the tabu list size: $|TL| = 2$ for instances with up to 20 customers and $|TL| = 7$ for instances with more than 20 customers. The solutions are kept in the tabu list until $|TL|$ moves involving the drone are performed. The problem-specific input parameters, such as drone endurance, service time and vehicles speed, are defined individually for each instance set (see Sections 6.2.1–6.2.3).

From all the neighborhood structures implemented, the most effective ones for all set of instances were *Shift(1,0)* and *Swap(1,1) intra-route*, which improved the solution by 78% and 43% of the times they were considered, respectively. The reinsertion moves, *Reinsertion* and *Or-opt2*, presented the worst performance when analyzed individually.

The tables presented hereafter employ the following notation. Column *Inst.* indicates the instance name, and *BKS* describes the best-known solution reported in the literature. Columns *Sol.*, *Gap* and *Time* indicate the best solution value, the gap between the best solution found by HTGVNS and BKS and the computational time in seconds, respectively. Finally, columns *Sol.* and *Gap* present the average solution cost of ten runs and the gap between the average solution cost of HTGVNS and the BKS.

HTGVNS uses the optimal TSP solution to create the initial FSTSP solution. The TSP solution is obtained using Concorde solver (Applegate et al., 2016) version 3.12.19 configured with CPLEX 12.6.3. Concorde is capable of finding the optimal solution without considerably increasing HTGVNS computational time. The computational experiments show that Concorde TSP Solver uses less than 7% of HTGVNS total time.

6.2.1. Results for Ponza's benchmark set

Ponza (2016) generated 50 instances that were split into two sets. A set of small instances considering between 5 to 10 customers, and a

Table 6
HTGVNS results for Ponza (2016) instances.

BKS		SA Ponza (2016)		HGVNS F&P (2020)			HTGVNS				
		Sol.	Time	Sol.	Sol.	Time	Sol.	Gap	Sol.	Gap	Time
050.1	11506.50	12518.93	213.87	11506.50	11857.02	4.39	11506.50	0.00	11979.39	4.11	6.98
050.2	10964.30	12475.14	208.36	10964.30	11049.21	4.41	10964.30	0.00	10984.98	0.19	5.38
050.3	11336.40	12664.65	191.04	11336.40	11336.40	4.03	11094.83	-2.13	11132.43	-1.80	5.14
050.4	10856.40	12908.18	184.85	10856.40	11929.50	5.02	10525.92	-3.04	10692.38	-1.51	5.98
050.5	10486.30	12164.83	189.86	10486.30	11034.30	4.52	10399.02	-0.83	10401.25	-0.81	4.35
100.1	15618.00	17974.85	267.42	15618.00	15623.84	11.94	15618.00	0.00	15832.94	1.38	15.49
100.2	14899.20	17342.18	272.35	14899.20	15127.50	9.43	14309.33	-3.96	14319.02	-3.89	14.76
100.3	14524.50	17181.88	265.45	14524.50	16074.42	10.93	14283.50	-1.66	14301.39	-1.54	14.30
100.4	15947.30	18538.03	266.75	15947.30	15947.30	10.34	15598.33	-2.19	15604.28	-2.15	13.67
150.5	14948.50	17407.43	312.77	14948.50	15479.22	9.94	14948.50	0.00	15048.43	0.67	17.29
150.1	19828.10	22823.38	365.04	19828.10	20069.32	20.84	19828.10	0.00	20042.43	1.08	18.84
150.2	20949.30	22549.55	383.72	20949.30	21390.32	25.05	20949.30	0.00	21132.30	0.87	27.44
150.3	22633.30	23114.14	379.99	22633.30	23108.49	29.97	22309.39	-1.43	22619.39	-0.06	21.39
150.4	20400.70	22651.00	382.67	20400.70	23390.90	25.39	20198.38	-0.99	20248.43	-0.75	27.20
150.5	22435.52	22807.41	384.69	22435.52	23032.05	31.84	22435.52	0.00	22798.66	1.62	26.78
200.1	25648.33	26991.21	456.74	25648.33	25983.49	30.44	25648.33	0.00	25700.32	0.20	34.83
200.2	27632.40	27848.14	452.88	27632.40	27985.35	71.39	25765.21	-6.76	25803.29	-6.62	51.39
200.3	26498.33	27143.78	510.11	26498.33	26837.33	70.38	25093.38	-5.30	25193.48	-4.92	57.98
200.4	28247.92	28503.18	517.44	28247.92	28463.95	87.98	26993.38	-4.44	27003.98	-4.40	73.83
200.5	24987.56	27875.87	515.30	24987.56	26357.49	69.38	24987.56	0.00	25204.44	0.87	53.08
Avg. Gap and Time			336.07			26.88		-1.64		-0.87	24.81

set of large instances including those with 50 to 200 customers. Five instances were generated for each number of customers.

For the set of small instances, HTGVNS resulted in solutions with the same objective value as the proposed formulation (see Table 5) in all executions. Therefore, HTGVNS always resulted in optimal solutions for all instances in the small set.

The results for the set of large instances are presented in Table 6. For some of these instances, Concorde was not able to find the initial solution. Therefore, for consistency, we used the Cheapest Insertion (CI) heuristic to obtain the initial solution for every test problem. The HTGVNS algorithm was able to improve the solution quality up to 15% when compared to Ponza (2016) and 6% compared to de Freitas and Penna (2020).

6.2.2. Results for Agatz benchmark set

A variant of the FSTSP is the Traveling Salesman Problem with Drones (TSP-D) introduced by Agatz et al. (2018). While, the FSTSP defines endurance (e) and service time (s^L and s^R), the TSP-D neglects those variables ($e = \infty$, $s^L = s^R = 0$), therefore, the drone can visit all customers. Furthermore, a drone trip can start and end at the same node ($launch = return$). The truck waits for the drone at the *launch* node or visits other customers then returns to the *launch* node. According to the authors, this strategy is beneficial, considering the truck can visit a node for either serve a customer, or supply the drone with another parcel. The authors proposed an enormous number of instances to the problem. In Bouman et al. (2015) it is explained how they are generated. They are divided into three sets, *uniform*, *single-center* and *double-center* distribution. The parameter α defines the ratio of drone speed to truck speed in an instance. Variable $\alpha = 1$ determines that the vehicles travel with the same speed, when $\alpha = 2$ the drone travels two times faster than the truck. Lastly, for $\alpha = 3$, the drone speed is three times as fast as the truck.

Tables 7–9 are broken down into scenarios stratified by the value of α . The results presented by these tables were collected by running the available code of the author's repository.⁴ It reflects the average of 10 runs for each instance size. As parameters, we adopted the same configuration employed in HTGVNS to obtain a comparable result. The TSP initial solution was obtained by the TSP solver Concorde. The four neighborhood structures available were applied: 2-point move (2p) that

swaps two nodes in the truck route, 2-opt move where two edges are removed and replaced with two new edges and the 1-point-move (1p) move where a node in the truck route is relocated to a new position. Finally, the last neighborhood combines all the previous moves.

According to the tables, it is possible to notice that HTGVNS improved the average solution for $\alpha = \{1, 2\}$ for both uniform and single-center combination. The HTGVNS could not improve the average solution of $\alpha = 3$ for double-center and single-center distribution, the later set achieved the worst improvement compared to the others distributions and values of α . Concerning computational time, the different distribution of customers does not affect runtime; however, when the vehicles present the same speed runtime increases. Agatz et al. (2018) exact partitioning algorithm outperforms HGVNS computational time for instances up to 100 customers. The scenario changes when the number of customers increases as HTGVNS presents a runtime much smaller than the LS.

Fig. 6, 7, and 8 show boxplot graphics within the solution value of 10 runs of HGVNS and HTGVNS. For each distribution (uniform, single-center, and double-center) exist one boxplot representing the instance size (10, 20, 50, 75, 100, 175, 250) of a certain α value. The thick line corresponds to the median, i.e., half of the values are below this line, divided into two quartiles, and the other half is above this line, divided into two quartiles. The crosses in the graph correspond to the outliers, values that present a considerable distance from the others. The plots show the tendency of the medium size instance in all distributions have a smaller gap between the minimal and maximum values. It is possible to notice that HTGVNS found better gaps than HGVNS; however, in some instances the gap is still considerable. Therefore, the algorithm still has a lot to be improved.

6.2.3. Results for Freitas and Penna (2020) benchmark set

The latest FSTSP benchmark set used to test the HTGVNS performance is the one proposed by de Freitas and Penna (2020). The authors introduced 25 FSTSP instances based on the TSPLIB Symmetric Traveling Salesman Problem instances (Skorobohatyj, 1995). These instances follow the FSTSP definition of Murray and Chu (2015). A different metric is considered to the vehicles travel distance. This is a reasonable consideration since drones are not affected by congestion, and they can fly in a straight line without considering the street network. On the other hand, trucks must respect traffic sign regulation and follow street network. The Euclidean metric is used to describe the drone

⁴ <https://github.com/pcbouman-eur/Drones-TSP>

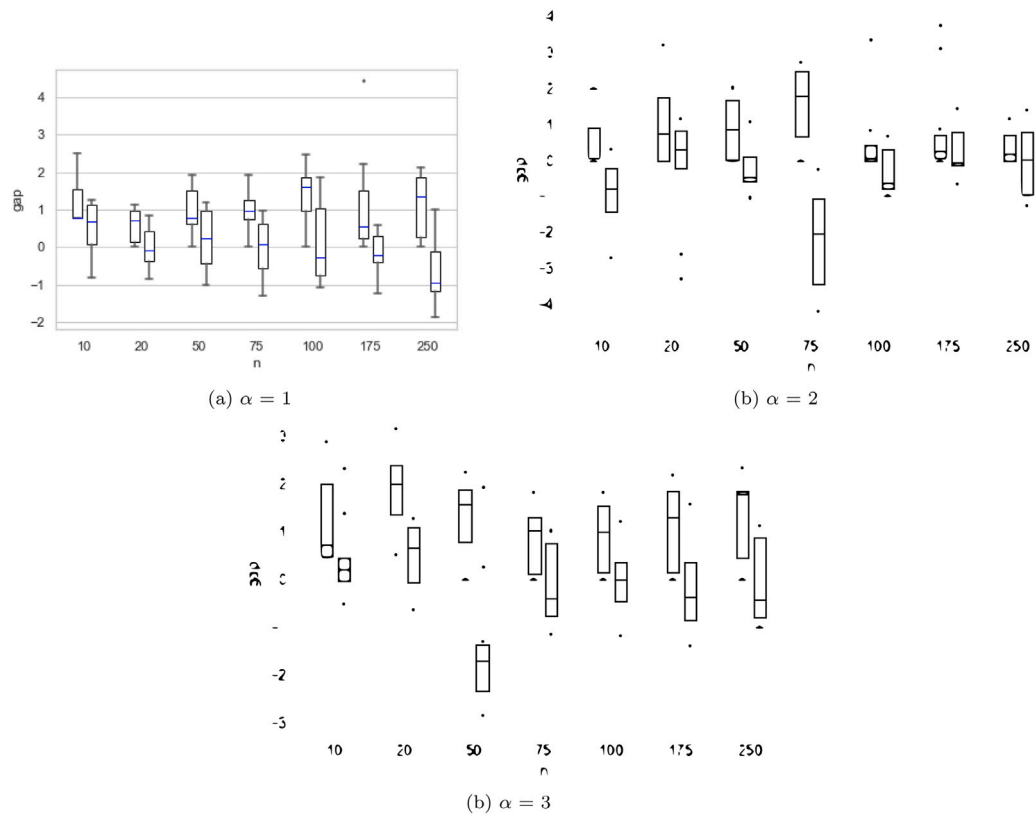


Fig. 6. Results for Agatz et al. (2018) instances with uniform distribution; left boxplots represent HGVNS results while right boxplots present HTGVNS results.

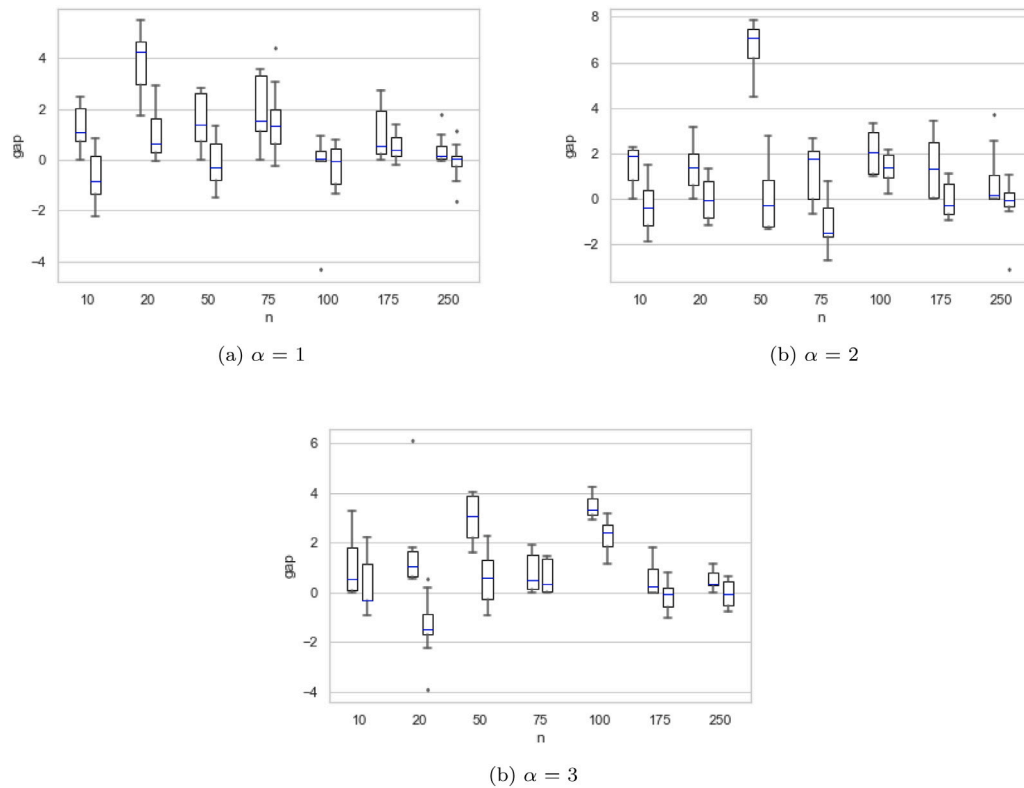


Fig. 7. Results for Agatz et al. (2018) instances with single-center distribution; left boxplots represent HGVNS results while right boxplots present HTGVNS results.

Table 7

HTGVNS results for Agatz et al. (2018) instances with uniform distribution.

n	BKS	LS Agatz et al. (2018)			HGVNS de Freitas and Penna (2020)			HTGVNS			
		<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	
$\alpha = 1$	10	286.82	286.82	0.00	0.00	289.82	1.05	0.14	289.82	1.05	0.15
	20	365.38	365.38	0.00	0.00	368.54	0.86	0.11	365.38	0.00	0.11
	50	550.38	550.38	0.00	0.90	559.20	1.60	3.71	551.49	0.20	3.53
	75	624.32	645.55	3.40	3.20	624.32	0.00	16.30	<u>624.26</u>	-0.01	16.30
	100	698.42	729.44	4.44	10.00	698.42	0.00	53.50	<u>696.15</u>	-0.33	53.75
	175	905.32	940.35	3.87	98.00	905.32	0.00	55.91	<u>903.89</u>	-0.16	57.14
	250	1113.96	1113.96	0.00	410.10	1135.32	1.92	185.19	<u>1078.00</u>	-3.23	185.19
	Average			1.67	74.60		0.78	44.98		-0.35	45.17
$\alpha = 2$	10	231.29	231.29	0.00	0.00	233.20	0.83	0.13	<u>228.00</u>	-1.42	0.13
	20	293.59	293.59	0.00	0.00	293.60	0.00	0.85	293.60	0.00	0.85
	50	420.80	428.63	1.86	1.20	420.80	0.00	2.30	420.80	0.00	2.32
	75	490.43	495.90	1.12	6.20	490.43	0.00	10.93	<u>459.71</u>	-6.26	10.95
	100	553.43	572.53	3.45	18.40	553.43	0.00	37.77	553.43	0.00	37.85
	175	704.53	722.83	2.60	177.20	704.53	0.00	39.28	704.53	0.00	40.65
	250	824.42	854.34	3.63	746.90	824.42	0.00	191.48	824.42	0.00	191.48
	Average			1.81	135.70		0.12	40.39		-1.10	40.60
$\alpha = 3$	10	210.42	210.42	0.00	0.00	215.88	2.59	0.13	211.49	0.51	0.14
	20	266.12	266.12	0.00	0.00	274.20	3.04	1.15	267.03	0.34	1.15
	50	389.80	391.96	0.55	1.60	389.80	0.00	2.19	<u>377.28</u>	-3.21	2.18
	75	447.64	453.30	1.26	8.00	447.64	0.00	10.95	443.30	-0.97	11.05
	100	510.20	530.53	3.98	25.67	510.20	0.00	37.26	510.20	0.00	37.13
	175	655.20	665.72	1.61	259.30	655.20	0.00	41.49	655.20	0.00	41.26
	250	758.32	785.86	3.63	1080.40	758.32	0.00	189.43	758.32	0.00	189.43
	Average			1.58	196.42		0.80	40.37		-0.48	40.34

Table 8

Results running HTGVNS in Agatz et al. (2018) with single-center distribution.

n	BKS	LS			HGVNS			HTGVNS			
		Agatz et al. (2018)			de Freitas and Penna (2020)						
		<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	
$\alpha = 1$	10	364.92	379.29	3.94	0.00	364.92	0.00	0.14	360.85	-1.11	0.14
	20	529.15	529.15	0.00	0.00	553.53	4.61	1.12	537.24	1.67	1.14
	50	763.28	763.28	0.00	0.30	784.32	2.76	3.93	756.20	-0.93	3.72
	75	978.32	1017.04	3.96	2.80	978.32	0.00	15.32	978.32	0.00	15.50
	100	1193.95	1203.42	0.79	8.50	1193.95	0.00	55.62	1193.95	0.00	56.64
	175	1629.32	1631.31	0.12	73.70	1629.32	0.00	53.34	1629.32	0.00	53.55
	250	1813.54	1857.16	2.41	358.40	1813.54	0.00	249.92	1813.54	0.00	229.92
	Average			1.60	63.39		1.05	54.20		-0.68	51.51
$\alpha = 2$	10	278.22	278.22	0.00	0.00	291.36	4.72	0.14	276.95	-0.46	0.14
	20	364.08	384.87	5.71	0.00	364.08	0.00	1.04	363.46	-0.17	1.04
	50	554.58	554.58	0.00	1.00	593.54	7.03	2.23	553.53	-0.19	2.24
	75	741.38	741.38	0.00	126.00	754.43	1.76	11.18	730.43	-1.48	43.36
	100	891.28	891.28	0.00	5.30	900.12	0.99	38.23	908.29	1.91	11.36
	175	1183.43	1208.94	2.16	15.70	1183.43	0.00	43.06	1183.12	-0.03	38.23
	250	1294.43	1396.24	7.87	136.40	1294.43	0.00	197.12	1290.38	-0.31	143.06
	Average			2.25	40.63		2.07	41.86		-0.10	34.20
$\alpha = 3$	10	238.20	238.20	0.00	0.00	242.20	1.68	0.13	237.43	-0.32	0.13
	20	315.27	315.27	0.00	0.00	319.80	1.44	0.86	310.67	-1.46	0.92
	50	474.82	474.82	0.00	1.00	493.43	3.92	2.19	478.32	0.74	2.26
	75	638.20	639.67	0.23	7.30	638.20	0.00	11.39	638.48	0.04	11.41
	100	777.25	777.25	0.00	20.80	804.43	3.50	37.74	798.54	2.74	38.40
	175	998.53	1067.54	6.91	211.60	998.53	0.00	41.20	990.42	-0.81	41.19
	250	1224.43	1232.56	0.66	883.80	1224.43	0.00	198.72	1224.13	-0.02	198.72
	Average			1.02	160.64		1.41	41.75		0.03	41.86

travel distance, and the Manhattan metric is adopted to simulate the city block, thus, determining the truck distance.

The eligible drone customers are randomly generated such that for every instance there are between 80% and 90% serviceable customers. Service times s^L and s^R are unitary. Moreover, both vehicles speed is 40 km/h, and the drone endurance is 40 min.

Table 10 provides the results obtained by HTGVNS. Note that column TSP Time indicates the runtime required to obtain the optimal TSP solution by Concorde. These results consider ten HTGVNS executions

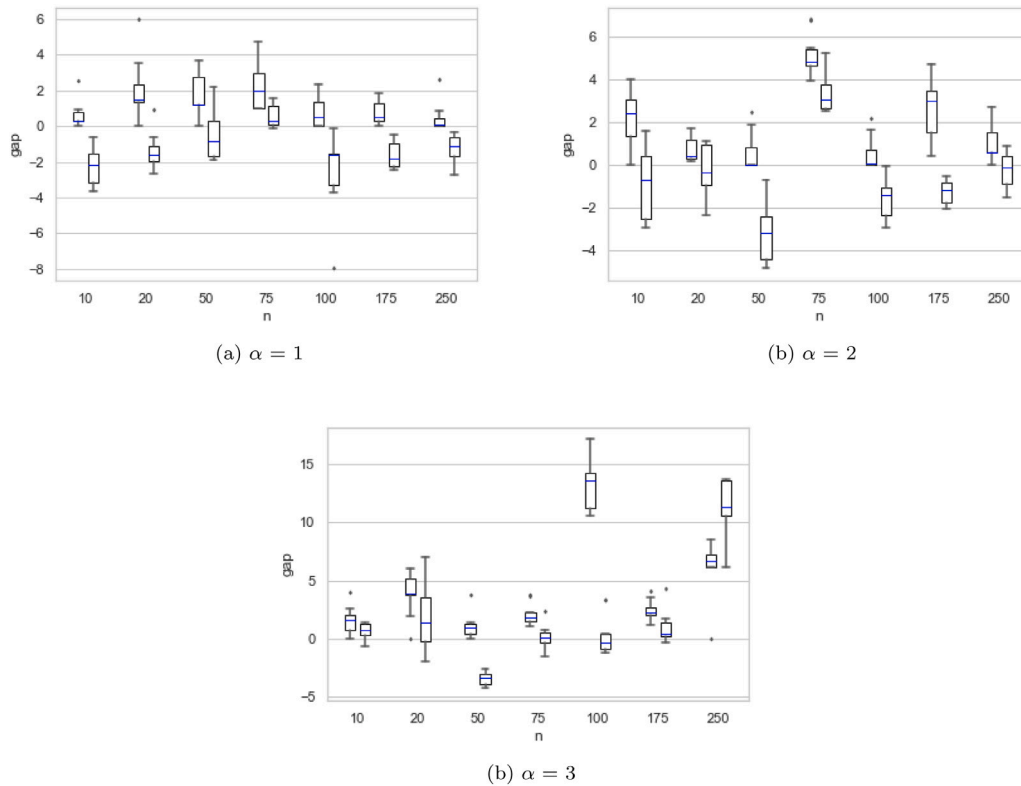
for each instance available online.⁵ Service time causes a large impact in instances with short travel times because the truck may need to wait a long time for the drone. Moreover, instances with customers disposed along a road are less impacted by the drone presence, since customers would be on the truck's way and it would be natural for the truck to

⁵ The instances and their solutions are available at <http://goal.ufop.br/fstsp/>.

Table 9

Results running HTGVNS in Agatz et al. (2018) with double-center distribution.

n	BKS	LS Agatz et al. (2018)			HGVNS de Freitas and Penna (2020)			HTGVNS			
		<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	<u>Sol.</u>	Gap.	Time	
$\alpha = 1$	10	634.42	652.30	2.82	0.00	634.42	0.00	0.15	615.80	-2.93	0.09
	20	754.81	776.32	2.85	0.00	754.81	0.00	1.39	733.88	-2.77	2.53
	50	1149.84	1149.84	0.00	0.40	1203.09	4.63	4.34	1153.29	0.30	4.95
	75	1430.96	1430.96	0.00	3.00	1494.57	4.45	19.39	1432.36	0.10	21.40
	100	1556.52	1605.50	3.15	9.60	1556.52	0.00	66.39	1516.36	-2.58	65.30
	175	2072.36	2155.85	4.03	88.30	2072.36	0.00	57.03	2030.44	-2.02	54.33
	250	2523.00	2572.80	1.97	353.60	2523.00	0.00	260.98	2493.64	-1.16	273.44
	Average			2.12	64.99		1.30	58.53		-1.58	60.29
$\alpha = 2$	10	490.64	490.64	0.00	0.00	510.09	3.96	0.13	487.24	-0.69	0.33
	20	603.31	603.31	0.00	0.00	605.04	0.29	0.98	602.73	-0.10	1.42
	50	850.31	864.21	1.63	1.00	850.31	0.00	2.24	811.08	-4.61	2.95
	75	1075.12	1075.12	0.00	5.30	1123.61	4.51	11.61	1109.53	3.20	15.39
	100	1199.09	1202.27	0.27	17.00	1199.09	0.00	38.22	1184.76	-1.20	43.49
	175	1611.13	1611.13	0.00	149.70	1659.37	2.99	42.31	1589.44	-1.35	50.39
	250	1862.44	1934.80	3.89	585.70	1862.44	0.00	193.17	1852.54	-0.53	249.39
	Average			0.83	108.39		1.68	41.24		-0.75	51.91
$\alpha = 3$	10	435.38	435.38	0.00	0.00	439.60	0.97	0.13	435.53	0.03	0.93
	20	567.49	567.49	0.00	0.00	586.74	3.39	0.99	573.94	1.14	1.44
	50	746.93	760.25	1.78	1.20	746.93	0.00	2.14	715.23	-4.24	2.90
	75	929.58	929.58	0.00	7.40	945.34	1.70	11.22	931.13	0.17	14.30
	100	1058.80	1058.80	0.00	23.10	1202.64	13.59	37.86	1051.00	-0.74	40.43
	175	1432.96	1432.96	0.00	212.90	1469.92	2.58	41.58	1439.44	0.45	48.33
	250	1635.12	1706.79	4.38	862.00	1635.12	0.00	196.27	1710.56	4.61	285.03
	Average			0.88	158.09		3.17	41.46		0.20	56.19

**Fig. 8.** Results for Agatz et al. (2018) instances with double-center distribution; left boxplots represent HGVNS results while right boxplots present HTGVNS results.

visit them. This behavior occurs in instance pr144 which prevents a significant improvement.

7. Conclusions

This work approaches a hybrid delivery system composed by truck and drone working collaboratively named Truck-Drone Delivery Problem (TDDP). The problem considers parcels distribution that seeks

to coordinate a traditional delivery truck with a drone that may be launched from the larger vehicle. This class of problem considers different vehicle speeds and drone endurance to minimize the required time to serve all customers. Here we studied two TDDPs variants. The Flying Sidekick Traveling Salesman Problem (FSTSP) presented by Murray and Chu (2015) and the Traveling Salesman Problem with Drone (TSP-D) introduced by Agatz et al. (2018).

Table 10
HTGVNS results for de Freitas and Penna (2020) instances.

Instance	n	HGVNS			HTGVNS					
		Sol.	Sol.	Time	Sol.	Gap	Sol.	Gap	TSP Time	Time
berlin52	52	210.03	220.23	6.51	204.40	-2.68	209.31	-0.33	0.01	4.20
bier127	127	3456.80	3587.88	53.74	3359.51	-2.81	3401.24	-1.55	0.05	66.32
ch130	130	178.16	180.40	44.22	168.50	-5.42	190.38	6.78	0.09	59.03
d198	198	461.83	461.83	67.84	452.55	-2.01	469.83	1.73	0.15	51.33
eil51	51	13.45	13.68	11.59	11.74	-12.74	11.93	-11.11	0.02	7.31
eil76	76	16.35	16.68	27.16	15.29	-6.47	16.11	-1.44	0.02	30.93
kroA100	100	587.80	609.71	30.99	575.26	-2.13	591.32	0.58	0.04	38.94
kroA150	150	764.42	780.93	41.02	739.49	-3.26	758.44	-0.77	0.07	38.33
kroA200	200	870.65	873.99	46.84	827.60	-4.94	869.43	-0.14	0.31	44.39
kroB150	150	763.15	773.72	50.48	713.63	-6.49	744.35	-2.43	0.28	43.39
kroB200	200	835.43	838.40	32.88	815.97	-2.33	831.43	-0.48	0.94	63.40
kroC100	100	658.38	660.93	36.66	619.94	-5.84	639.42	-2.87	0.03	48.12
kroD100	100	606.45	652.34	40.21	570.98	-5.85	593.43	-2.00	0.06	54.80
kroE100	100	651.31	659.48	48.74	620.68	-4.70	639.32	-1.82	0.17	60.13
lin105	105	378.25	380.43	40.33	377.46	-0.21	382.55	1.13	0.06	61.32
pr107	107	1204.42	1224.35	32.56	1128.71	-6.29	1198.64	-0.47	0.02	52.40
pr124	124	1653.80	1996.62	25.67	1632.82	-1.27	1785.73	6.61	0.22	49.30
pr136	136	2642.00	2789.00	45.13	2595.17	-1.77	2639.32	-0.10	0.63	52.39
pr144	144	1666.25	1675.75	43.42	1666.25	0.00	1707.47	2.46	0.09	48.20
pr152	152	2114.04	2128.53	61.51	2082.03	-1.51	2187.58	3.45	0.22	70.33
rat99	99	37.15	37.33	35.41	33.93	-8.67	37.65	1.34	0.04	39.87
rat195	195	71.40	71.93	46.78	67.17	-5.92	69.42	-2.75	1.89	70.77
rd100	100	240.46	243.84	34.69	224.11	-6.80	236.42	-1.66	0.82	43.59
st70	70	20.50	21.00	3.86	17.14	-16.40	22.95	11.67	0.01	10.83
Average			37.58		-4.86	0.26	46.23			

A Mixed Integer Programming formulation was proposed, solving multiple previously unsolved instances with up to 10 customers. In addition to providing new optimal solutions, it also provided stronger bounds than those obtained from formulations previously proposed for the FSTSP (Murray and Chu, 2015; Ponza, 2016). However, due to the NP-hard characteristic of the problem, only small-scale instances could be solved.

We introduced a metaheuristic to tackle the TDDPs large-sized instances. The Hybrid Tabu General Variable Neighborhood Search (HTGVNS) algorithm initially constructs a solution by using a TSP solver to build the truck tour. This solution is then improved by the metaheuristic General Variable Neighborhood Search (GVNS) mixed with a Tabu Search list to avoid cycling.

The solution approach was validated through numerical analysis that indicates the potential of the truck-drone delivery system to improve the operation. The HTGVNS was evaluated considering three benchmark sets. For the FSTSP sets, the algorithm improved all the best-known solutions. On average, these solutions were improved by 3.25%. For the TSP-D instances, the heuristic improved or achieved the same result for 74% of the instance set.

The numerical analysis indicated that the hybrid delivery system can be more efficient by employing drones traveling faster than trucks. For the TSP-D benchmark set, the best solution found considers instances in which the drone travels twice as fast as the truck, and the smallest improvements were observed for instances where both vehicles present the same speed.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the National Council for Scientific and Technological Development (CNPq), Grant 438473/2018-3. We also thank the Federal University of Ouro Preto (UFOP), and the Coordination for the Improvement of Higher Education Personnel (CAPES), Finance Code 001, for supporting the development of the present study.

References

- Agatz, N., Bouman, P., Schmidt, M., 2018. Optimization approaches for the traveling salesman problem with drone. *Transp. Sci.* 52 (4), 965–981. <http://dx.doi.org/10.1287/trsc.2017.0791>.
- Applegate, D., Bixby, R., Chvatal, V., Cook, W., 2016. CONCORDE TSP solver. URL <http://www.math.uwaterloo.ca/tsp/concorde.html>. (Accessed: 07 June 2018).
- Arenzana, A.O., Macias, J.J.E., Angeloudis, P., 2020. Design of hospital delivery networks using unmanned aerial vehicles. *Transp. Res. Rec.* 2674 (5), 405–418. <http://dx.doi.org/10.1177/0361198120915891>.
- Beasley, J., 1983. Route first—Cluster second methods for vehicle routing. *Omega* 11 (4), 403–408. [http://dx.doi.org/10.1016/0305-0483\(83\)90033-6](http://dx.doi.org/10.1016/0305-0483(83)90033-6), URL <https://www.sciencedirect.com/science/article/pii/0305048383900336>.
- Bouman, P., Agatz, N., Schmidt, M., 2015. Instances for the TSP with drone. <http://dx.doi.org/10.5281/ZENODO.22245>.
- Bouman, P., Agatz, N., Schmidt, M., 2018. Dynamic programming approaches for the traveling salesman problem with drone. *Networks* 72 (4), 528–542. <http://dx.doi.org/10.1002/net.21864>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21864>.
- Croes, G.A., 1958. A method for solving traveling-salesman problems. *Oper. Res.* 6 (6), 791–812, URL <http://www.jstor.org/stable/167074>.
- Dayarian, I., Savelsbergh, M., Clarke, J.-P., 2020. Same-day delivery with drone resupply. *Transp. Sci.* 54 (1), 229–249. <http://dx.doi.org/10.1287/trsc.2019.0944>.
- Dell'Amico, M., Montemanni, R., Novellani, S., 2020. Matheuristic algorithms for the parallel drone scheduling traveling salesman problem. *Ann. Oper. Res.* 289, 211–226. <http://dx.doi.org/10.1007/s10479-020-03562-3>, URL <https://link.springer.com/article/10.1007/s10479-020-03562-3>.
- Dorling, K., Heinrichs, J., Messier, G.G., Magierowski, S., 2017. Vehicle routing problems for drone delivery. *IEEE Trans. Syst. Man Cybern. A* 47 (1), 70–85. <http://dx.doi.org/10.1109/TSMC.2016.2582745>.
- Douglas, A., 2020. Flytrex takes-off with drone deliveries following COVID-19 social distancing protocols. URL <https://bit.ly/3dzQD48>.
- Ferrandez, S., Harbison, T., Weber, T., Sturges, R., Rich, R., 2016. Optimization of a truck-drone in tandem delivery network using k-means and genetic algorithm. *J. Ind. Eng. Manag.* 9, 374. <http://dx.doi.org/10.3926/jiem.1929>.
- de Freitas, J.C., Penna, P.H.V., 2020. A variable neighborhood search for flying sidekick traveling salesman problem. *Int. Trans. Oper. Res.* 27 (1), 267–290. <http://dx.doi.org/10.1111/itor.12671>.
- Glover, F., Laguna, M., 1998. Tabu search. In: *Handbook of Combinatorial Optimization*. 1–3, Springer US, pp. 2093–2229. http://dx.doi.org/10.1007/978-1-4613-0303-9_33.
- Gonzalez-R, P.L., Canca, D., Andrade-Pineda, J.L., Calle, M., Leon-Blanco, J.M., 2020. Truck-drone team logistics: A heuristic approach to multi-drop route planning. *Transp. Res. C* 114, 657–680. <http://dx.doi.org/10.1016/j.trc.2020.02.030>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X19309568>.
- Guillot, G., 2020. Using Drones for Food Deliveries, Social Distancing, and Health Monitoring: Drones in America MarketScale. URL <https://bit.ly/2LnyzOO>.

- Ha, Q.M., Deville, Y., Pham, Q.D., Hà, M.H., 2018. On the min-cost traveling salesman problem with drone. *Transp. Res. C* 86, 597–621. <http://dx.doi.org/10.1016/j.trc.2017.11.015>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X17303327>.
- Hansen, P., Mladenović, N., Pérez, J.A.M., 2010. Variable neighbourhood search: Methods and applications. *4OR* 175, 367–407. <http://dx.doi.org/10.1007/s10479-009-0657-6>.
- Hansen, P., Mladenović, N., Todosijević, R., Hanafi, S., 2016. Variable neighborhood search: basics and variants. *EURO J. Comput. Opt.* 5, <http://dx.doi.org/10.1007/s13675-016-0075-x>.
- Jeong, H.Y., Song, B.D., Lee, S., 2019. Truck-drone hybrid delivery routing: Payload-energy dependency and No-Fly zones. *Int. J. Prod. Econ.* 214, 220–233. <http://dx.doi.org/10.1016/j.ijpe.2019.01.010>, URL <https://www.sciencedirect.com/science/article/pii/S0925527319300106>.
- Karak, A., Abdelghany, K., 2019. The hybrid vehicle-drone routing problem for pick-up and delivery services. *Transp. Res. C* 102, 427–449. <http://dx.doi.org/10.1016/j.trc.2019.03.021>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X18312932>.
- Mercer, C., 2018. How are drones used? Top companies using drones right now. <https://www.techworld.com/picture-gallery/apps-wearables/best-uses-of-drones-3605145/>. (Accessed 30 May 2018).
- Moshref-Javadi, M., Hemmati, A., Winkenbach, M., 2020. A truck and drones model for last-mile delivery: A mathematical model and heuristic approach. *Appl. Math. Model.* 80, 290–318. <http://dx.doi.org/10.1016/j.apm.2019.11.020>, URL <https://www.sciencedirect.com/science/article/pii/S0307904X19306936>.
- Murray, C.C., Chu, A.G., 2015. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transp. Res. C* 54, 86–109. <http://dx.doi.org/10.1016/j.trc.2015.03.005>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X15000844>.
- Murray, C.C., Raj, R., 2020. The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones. *Transp. Res. C* 110, 368–398. <http://dx.doi.org/10.1016/j.trc.2019.11.003>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X19302505>.
- Otto, A., Agatz, N., Campbell, J., Golden, B., Pesch, E., 2018. Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey. *Networks* 72 (4), 411–458. <http://dx.doi.org/10.1002/net.21818>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21818>.
- Penna, P.H.V., Subramanian, A., Ochi, L.S., 2013. An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. *J. Heuristics* 19 (2), 201–232. <http://dx.doi.org/10.1007/s10732-011-9186-y>.
- Poikonen, S., Wang, X., Golden, B., 2017. The vehicle routing problem with drones: Extended models and connections. *Networks* 70 (1), 34–43. <http://dx.doi.org/10.1002/net.21746>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21746>.
- Ponza, A., 2016. Optimization of drone-assisted parcel delivery (Ph.D. thesis). Università Degli Studi Di Padova, <http://dx.doi.org/10.13140/RG.2.2.24444.56962>.
- Pugliese, L.D.P., Guerriero, F., 2017. Last-mile deliveries by using drones and classical vehicles. In: Sforza, A., Sterle, C. (Eds.), *Optimization and Decision Science: Methodologies and Applications*. Springer International Publishing, pp. 557–565.
- Saleu, R.G.M., Deroussi, L., Feillet, D., Grangeon, N., Quilliot, A., 2018. An iterative two-step heuristic for the parallel drone scheduling traveling salesman problem. *Networks* 72 (4), 459–474. <http://dx.doi.org/10.1002/net.21846>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21846>.
- Schermer, D., Moeini, M., Wendt, O., 2019. A matheuristic for the vehicle routing problem with drones and its variants. *Transp. Res. C* 106, 166–204. <http://dx.doi.org/10.1016/j.trc.2019.06.016>, URL <https://www.sciencedirect.com/science/article/pii/S0968090X19301081>.
- Skorobohatyj, G., 1995. TSPLIB. URL <https://www.iwr.uni-heidelberg.de/groups/comopt/>.
- Song, B.D., Park, K., Kim, J., 2018. Persistent UAV delivery logistics: MILP formulation and efficient heuristic. *Comput. Ind. Eng.* 120, 418–428. <http://dx.doi.org/10.1016/j.cie.2018.05.013>, URL <https://www.sciencedirect.com/science/article/pii/S0360835218302146>.
- Souza, M.J.F., Coelho, I.M., Ribas, S., Santos, H.G., Merschmann, L.H.C., 2010. A hybrid heuristic algorithm for the open-pit-mining operational planning problem. *European J. Oper. Res.* 207 (2), 1041–1051. <http://dx.doi.org/10.1016/j.ejor.2010.05.031>.
- Ulmer, M.W., Thomas, B.W., 2018. Same-day delivery with heterogeneous fleets of drones and vehicles. *Networks* 72 (4), 475–505. <http://dx.doi.org/10.1002/net.21855>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21855>.
- Wang, X., Poikonen, S., Golden, B., 2017. The vehicle routing problem with drones: Several worst-case results. *Optim. Lett.* 11, <http://dx.doi.org/10.1007/s11590-016-1035-3>.